

Batman Attempts

Yunru Zheng

January 24, 2026

1 Justification on Constraints

I'm trying to show that constraints are necessary and very important for our final result here. First of all, I started with learning rate=0.0001 and $\theta = [1.5, 0.4, 1.0, 0.2]$ and since it has a nice starting shape like a batman and I'm trying to increase the ears.

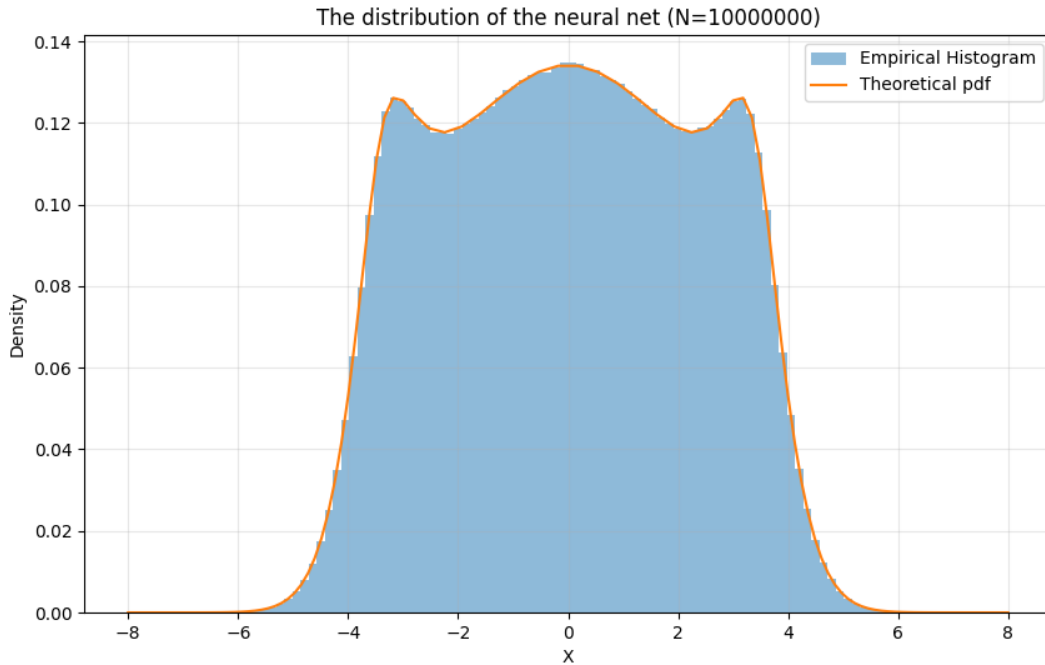


Figure 1: The distribution of the neural net for $\theta = [1.5, 0.4, 1.0, 0.2]$.

The first attempt is no restriction. The variance graph shows that my algorithm indeed decreases the variance if no constraints are applied.

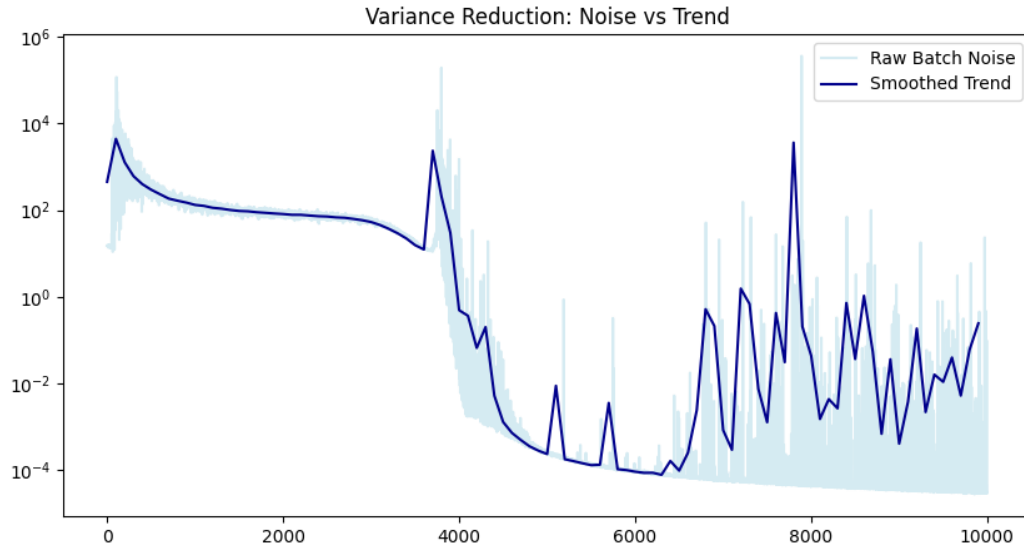


Figure 2: The variance change starting from $\theta = [1.5, 0.4, 1.0, 0.2]$.

However, if we see the neural net graph, the slope points negatively, which isn't what we want. Some kind of constraints are necessary.

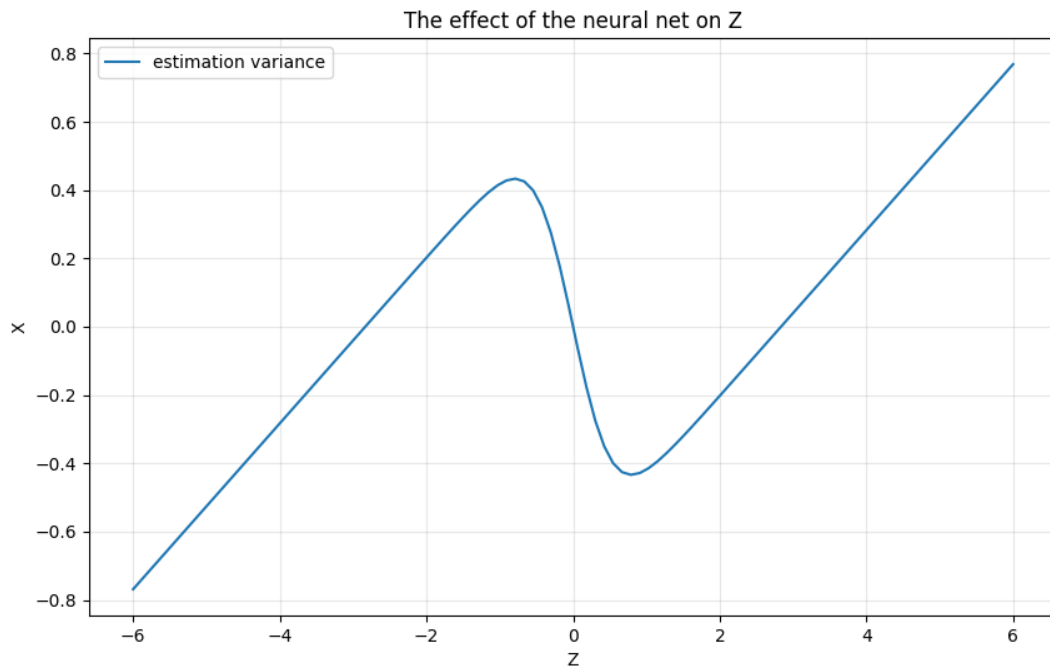


Figure 3: The neural net for $\theta = [1.18, 0.51, -0.32, 0.24]$.

2 First attempt on constraints

The first few constraints that I set up was:

- $b, d > 0$ since the width is certainly positive, and we want the slope after the turn to be smaller than the initial slope
- $2a - b/d > 0$ to ensure that the neural net is one-to-one.

This also implies that $a > 0$ so it prevents the previous case when the initial slope is negative. I did not restrict c intentionally to see what c value the optimizer prefers. The actual application is as follows:

```
# Attempt 1: b, d > 0, 2a - b/d > 0
b_new = jnp.maximum(b_new, 0.1)
d_new = jnp.maximum(d_new, 0.1)

min_a = 0.1 + b_new / (2.0 * d_new)
a_new = jnp.maximum(a_new, min_a)
```

Figure 4: Applied first set of constraints.

This doesn't give satisfactory results either, not just the variance not being reduced. From the neural net, we see that it's one-to-one, but if we inspect closer, $c = 0.035$ is so tiny that turning when Z still has so much weight isn't a good choice, however, by doing this the optimizer makes sure that with $a = 15.9$, a very high slope, most of the weight of X is around 0, which makes the variance smaller. As you can see from Figure 7, instead of generating smooth batman ears, it spikes at around ± 1.3 .

Another evidence that the optimization is wrong is that the estimate $E[X^4]$ varies around 1 instead of being around the known value 3.

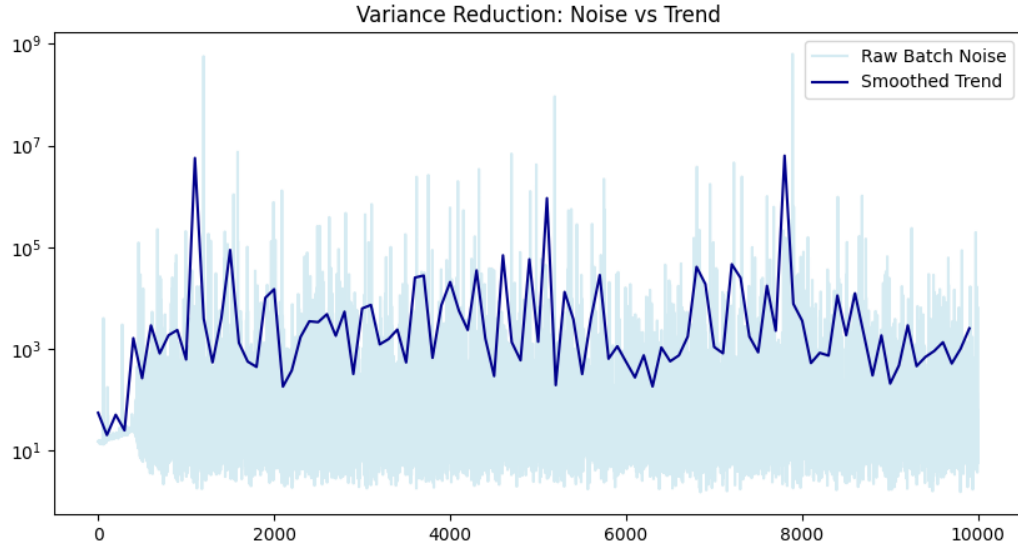


Figure 5: The variance change starting from $\theta = [1.5, 0.4, 1.0, 0.2]$ after minor constraints.

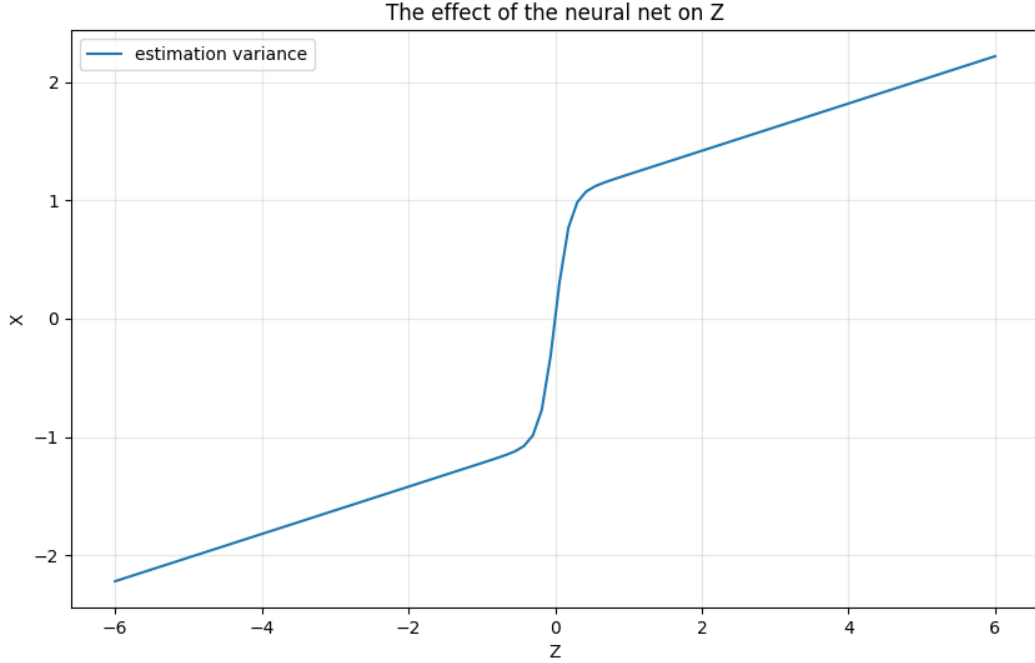


Figure 6: The neural net for $\theta = [15.900, 3.150, 0.035, 0.100]$.

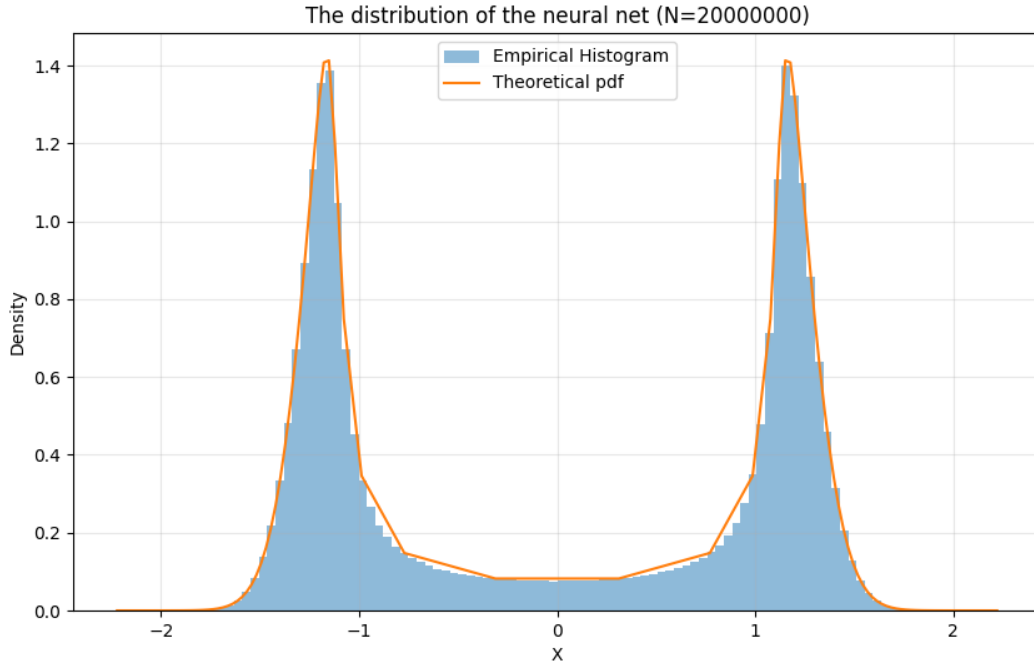


Figure 7: The distribution of the neural net for $\theta = [15.900, 3.150, 0.035, 0.100]$.

3 Second attempt on constraints

In fact one factor I didn't mention is the impact of restrictions on $2a - b/d$, which might restrict a or b instead. Since we want the tail to behave normally, instead of just set it to be greater than 0, I set it to be $2a - b/d > 1$. The real question comes: Should it restrict a or b ? I decided to try a first. I started with $\theta = 2.0, 0.5, 1.0, 0.2$, learning rate 0.0005. Here is my actual code implementation:

```

# Attempt 2: b, d > 0, c >= 0.5, a <=5, 2a - b/d > 1 restricts b
# (so the tail doesn't cluster like crazy)

d_new = jnp.maximum(d_new, 0.2)

c_new = jnp.maximum(c_new, 0.5)

min_a = 0.5 + b_new / (2.0 * d_new)
a_new = jnp.maximum(a_new, min_a)

# Restrict a instead of b
b_new = jnp.maximum(b_new, 0.1)

```

Figure 8: Constraint set 2

If you remember my last report and the diabolical variance reduction graph with variance increasing, yes, because I was restricting a ! Here you see the same type of increasing variance, because I forced a to go up based on my constraint, despite it wanting to go down.

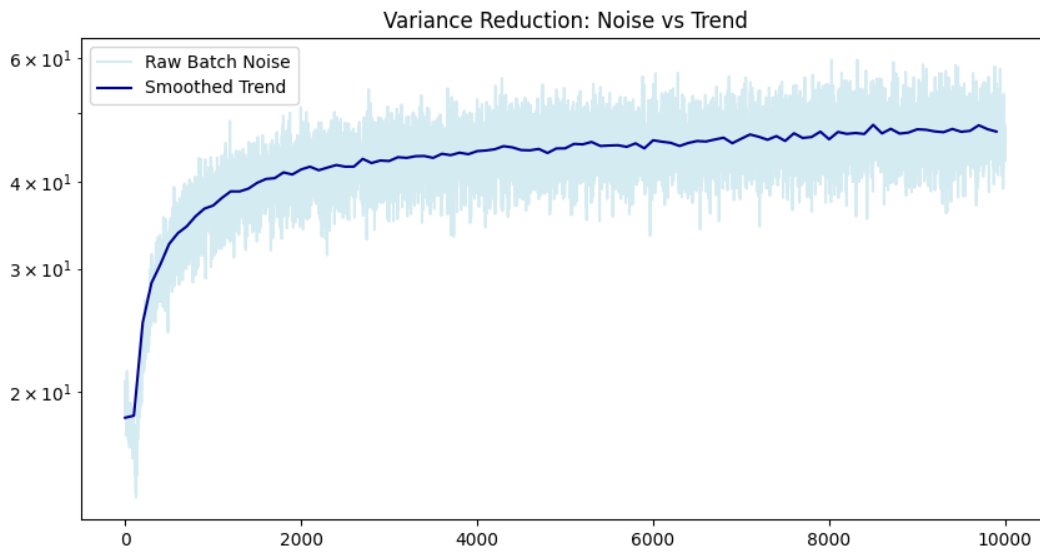


Figure 9: The variance change starting from $\theta = [1.5, 0.4, 1.0, 0.2]$ after minor constraints.

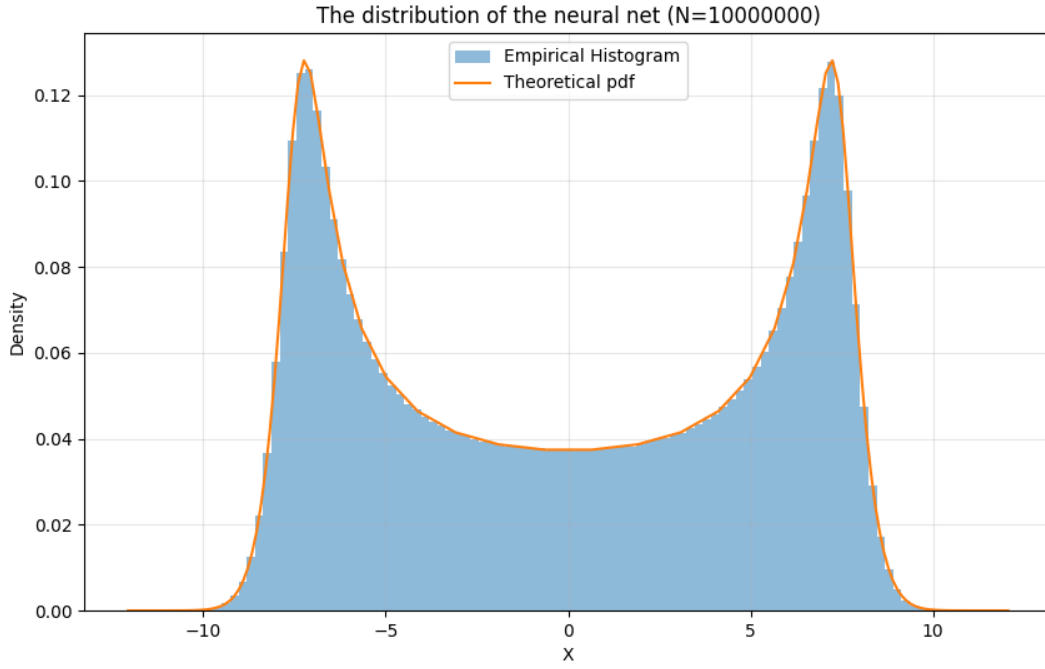


Figure 10: The distribution of the neural net for $\theta = [6.56, 2.75, 0.50, 0.23]$.

```

Iter 0: Loss=20.6366 | Theta=[1.9976407 0.50070727 0.9985993 0.20068647]
Iter 0: Loss=20.6366 | Estimate=3.0286
Iter 1000: Loss=37.7808 | Theta=[4.973364 1.7893455 0.5 0.2 ]
Iter 1000: Loss=37.7808 | Estimate=3.0824
Iter 2000: Loss=43.5618 | Theta=[5.5105314 2.0176282 0.5 0.20133874]
Iter 2000: Loss=43.5618 | Estimate=3.0870
Iter 3000: Loss=42.9121 | Theta=[5.7594123 2.1612926 0.5 0.20546901]
Iter 3000: Loss=42.9121 | Estimate=3.0004
Iter 4000: Loss=47.1099 | Theta=[5.9315934 2.2771664 0.5 0.20962231]
Iter 4000: Loss=47.1099 | Estimate=3.1138
Iter 5000: Loss=39.8024 | Theta=[6.066374 2.3767476 0.5 0.21349156]
Iter 5000: Loss=39.8024 | Estimate=2.7312
Iter 6000: Loss=41.4799 | Theta=[6.182534 2.466972 0.5 0.21706618]
Iter 6000: Loss=41.4799 | Estimate=2.7950
Iter 7000: Loss=47.0085 | Theta=[6.291393 2.5478272 0.5 0.21996671]
Iter 7000: Loss=47.0085 | Estimate=3.0481
Iter 8000: Loss=38.4553 | Theta=[6.4006934 2.6213555 0.5 0.22212267]
Iter 8000: Loss=38.4553 | Estimate=2.6253
Iter 9000: Loss=51.3707 | Theta=[6.478393 2.6894352 0.5 0.22492962]
Iter 9000: Loss=51.3707 | Estimate=3.1805

Final Theta:, ['6.56e+00', '2.75e+00', '5.00e-01', '2.27e-01'], learning rate: 0.0005

```

Figure 11: The record of iterations and parameter change during attempt 2.

4 Third attempt on constraints (Not really just the third but the third that I have some tangible results)

Now, instead of restricting a , I restrict b . I started with $\theta = 2.0, 0.5, 1.0, 0.2$, learning rate 0.0005. Here is my actual code implementation:

```
d_new = jnp.maximum(d_new, 0.05)
c_new = jnp.maximum(c_new, 0.5)
a_new = jnp.clip(a_new, 1.0, 3.0)
# Restrict b instead of a
max_b = d_new * (2.0 * a_new - 1.0)
b_new = jnp.clip(b_new, 0.0, max_b)
```

Figure 12: Constraint set 3.

This is a bit more satisfactory than the previous attempt at restricting the parameter a . Let's check the basic standard for a passable batman: The variance indeed decreases, there's a turn in the neural net, and there's a visible batman ear in the distribution graph. However, it's still very problematic if we examine the actual report.

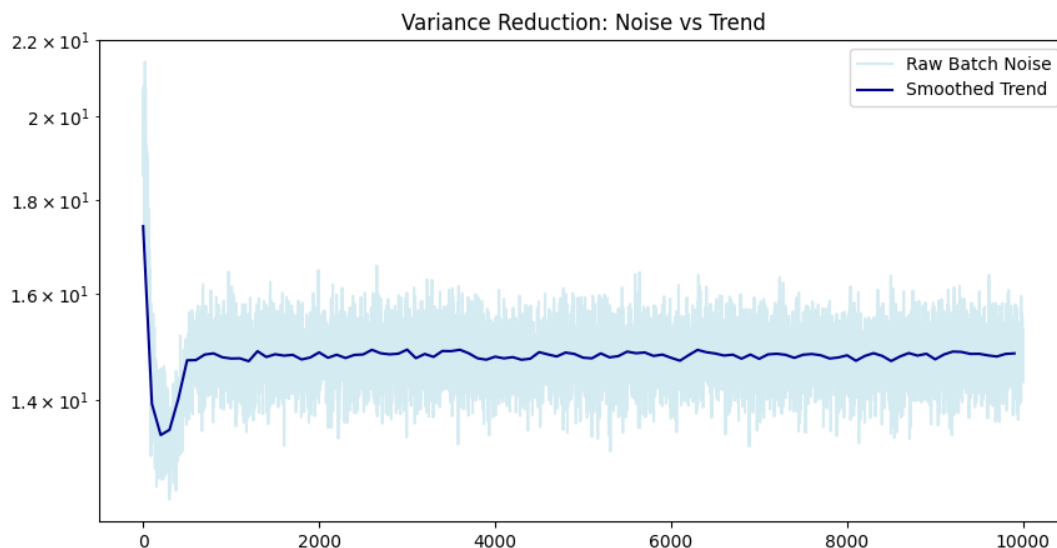


Figure 13: The variance change starting from $\theta = [1.5, 0.4, 1.0, 0.2]$ after minor constraints.

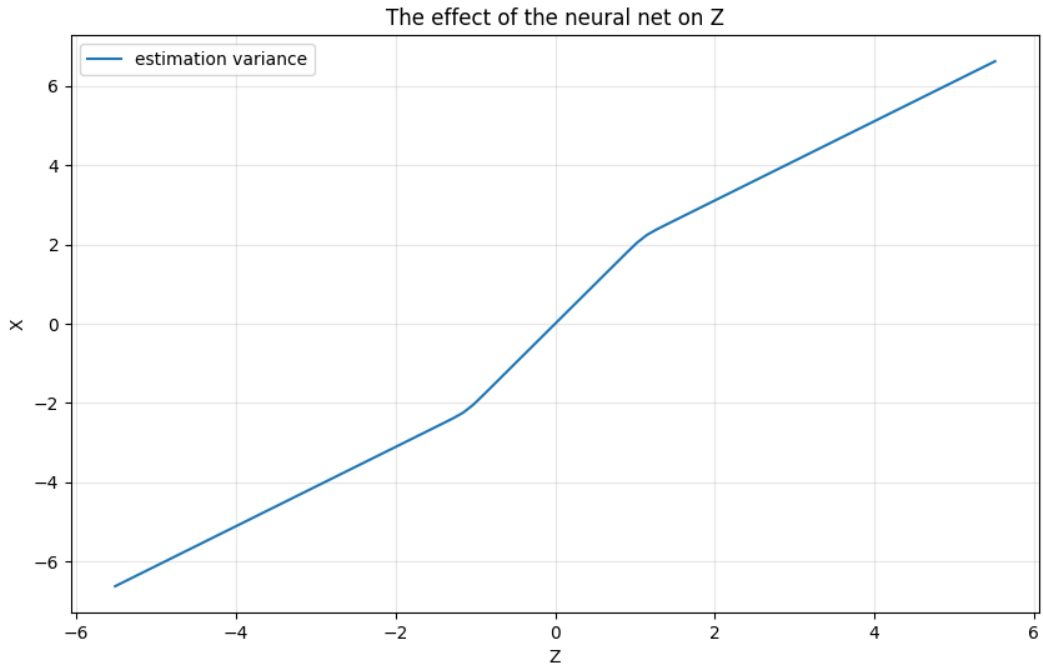


Figure 14: The neural net for $\theta = [1.00, 0.05, 1.11, 0.05]$.

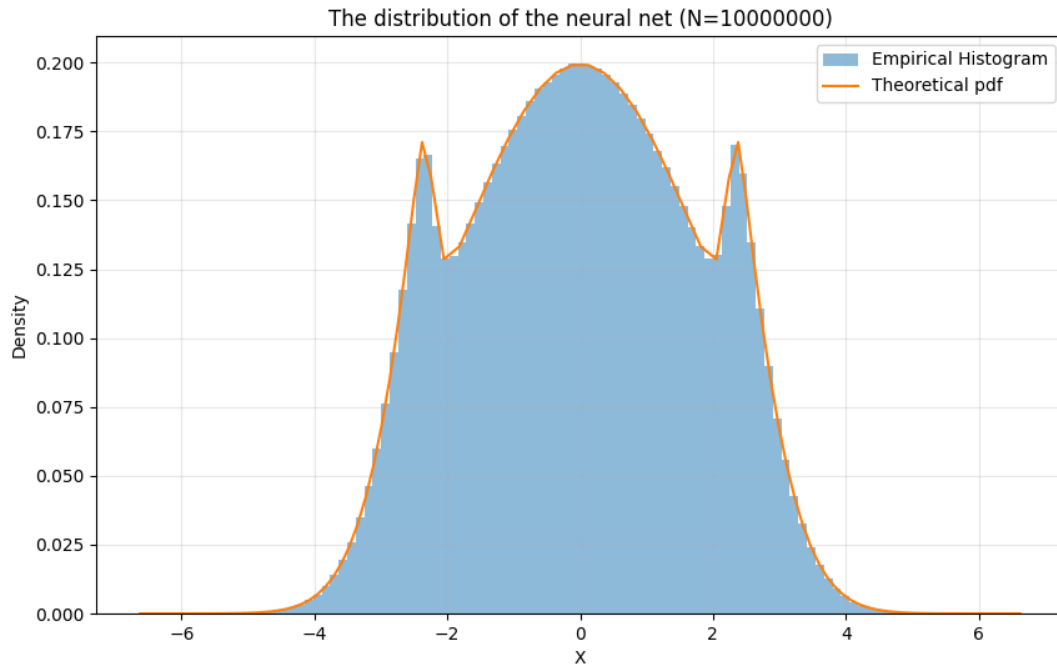


Figure 15: The distribution of the neural net for $\theta = [1.00, 0.05, 1.11, 0.05]$.

Despite our step size being very small, 0.0005, after 1000 iterations, a quickly drops down to 1, which is what the optimizer is heading regardless of other parameters (since we didn't restrict a , it goes down instead of up). This is in general not good, since our intention is to shift the center of the standard Gaussian far from the center, which requires $a > 1$. a is basically hugging a wall since I restricted it to be between 1.0 and 3.0. This further restricts my b and d , so for the rest of the iteration it just repeats itself. I did it again with $1.5 \leq a \leq 3.0$, but then a just hugs the wall at 1.5 and the graph changes. This is in no way the correct way to find an optimized solution.

```

Iter   0: Loss=20.6366 | Theta=[1.9976407  0.50070727 0.9985993  0.20068647]
Iter   0: Loss=20.6366 | Estimate=3.0286
Iter 1000: Loss=14.5320 | Theta=[1.          0.05000578 1.0816777  0.05000578]
Iter 1000: Loss=14.5320 | Estimate=2.9568
Iter 2000: Loss=15.0811 | Theta=[1.          0.05044995 1.1037065  0.05044995]
Iter 2000: Loss=15.0811 | Estimate=3.0376
Iter 3000: Loss=15.2654 | Theta=[1.          0.05009234 1.1122372  0.05009234]
Iter 3000: Loss=15.2654 | Estimate=3.0543
Iter 4000: Loss=14.9234 | Theta=[1.          0.05048379 1.1088269  0.05048379]
Iter 4000: Loss=14.9234 | Estimate=3.0054
Iter 5000: Loss=15.2345 | Theta=[1.          0.05018563 1.1108624  0.05018563]
Iter 5000: Loss=15.2345 | Estimate=3.0537
Iter 6000: Loss=14.6914 | Theta=[1.          0.05          1.1108905  0.05          ]
Iter 6000: Loss=14.6914 | Estimate=2.9958
Iter 7000: Loss=14.5468 | Theta=[1.          0.05028113 1.1106822  0.05028113]
Iter 7000: Loss=14.5468 | Estimate=2.9326
Iter 8000: Loss=15.4974 | Theta=[1.          0.05049301 1.1101747  0.05049301]
Iter 8000: Loss=15.4974 | Estimate=3.1388
Iter 9000: Loss=15.0107 | Theta=[1.          0.05009064 1.1081572  0.05009064]
Iter 9000: Loss=15.0107 | Estimate=3.0247

Final Theta:, ['1.00e+00', '5.06e-02', '1.11e+00', '5.06e-02'], learning rate: 0.0005

```

Figure 16: The record of iterations and parameter change during attempt 3.

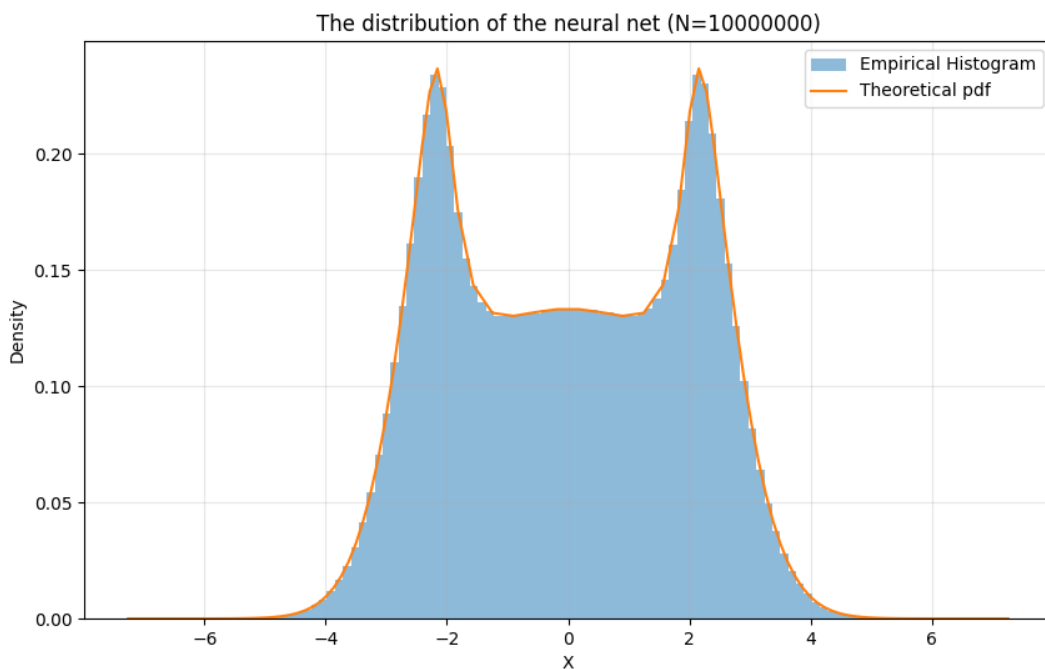


Figure 17: $1.5 \leq a \leq 3.0$. The distribution of the neural net for $\theta = [1.50, 0.20, 0.63, 0.10]$.

5 Conclusion

Based on the two cases when I restrict a or b , the optimizer pushes a to be smaller and b to be bigger when unrestricted, which means that $2a - b/d < 0$ if we don't put any restrictions. However, when we put restrictions on a or b , it either generates something with worse variance, or stays within the boundary limit but sticks to the boundary. This is the next problem that I need to solve.